


Designed by Remus Teodorescu 2026

LECTURE 13

# Mini-Projects

Twenty-three projects across the ten exercise sets of Part 2, in pairs: one forward on richer physics, one inverse from committed readings. Each takes its exercise further, and none has its answer written down in advance.

Deep Learning for Engineering · Aalborg University

*One project per team of two or three · chosen after L12.2*

# What a Mini-Project Is, and Is Not

## WHAT MAKES IT DIFFERENT FROM AN EXERCISE

- the answer is not known in advance, to you or to me
- you choose what to model and what to leave out, and defend the choice
- a negative result is a result, provided you can say why
- and the deliverable argues rather than reports

### WHAT EVERY PROJECT NEEDS

| | |
|---|---|
| **a question** | stated before you start |
| **a baseline** | classical, and taken seriously |
| **a held-out test** | chosen to be hard, not convenient |
| **a limit** | where your method stops working |

### THE BASELINE IS NOT OPTIONAL

Part 2 named cases where a classical method wins. Say whether this is one.

### TEAMS OF TWO OR THREE

One project per team. Come and talk before you commit to one.

# Heat Transfer

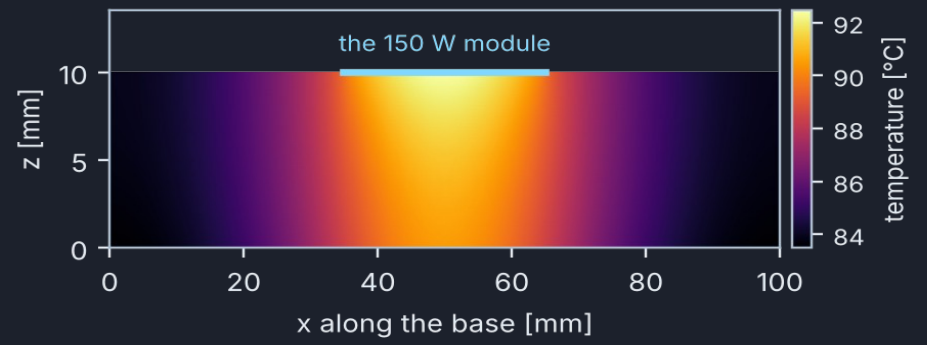
## WHERE THE NETWORK BEATS THE CLASSICAL METHOD

| | |
|---|---|
| **identification** | $h_{eff}$ from one thermocouple, 5 % low |
| **late transient** | within 0.06 K at 150 s |
| **built-in physics** | hand calculation and balance in the loss |
| **reuse** | trained once, each answer under a ms |

## WHERE THE CLASSICAL METHOD STAYS AHEAD

One forward solve: finite differences meet 0.01 K in 4 ms steady, 0.05 K in 3 s for the pulse. The network's case is the inverse problem and the many-question study, not one solve.

## THE FIELD THE NETWORK LEARNS




*the temperature of the heat-sink base under the 150 W module, hottest point 92.45 °C*

**1 thermocouple**

beside the module identifies the film coefficient of the whole fin array, 5 % low (Ex08, part 2)

# Fluid Flow

## WHERE THE NETWORK BEATS THE CLASSICAL METHOD

| | |
|---|---|
| **accuracy** | 2 to 6·10$^{-6}$ of the steam fraction |
| **against any grid** | 25 to 120× more exact than 401 × 41 |
| **design by gradient** | Newton finds 0.3174 m/s, exact |
| **many dimensions** | the grid pays ×4 per halving of h |

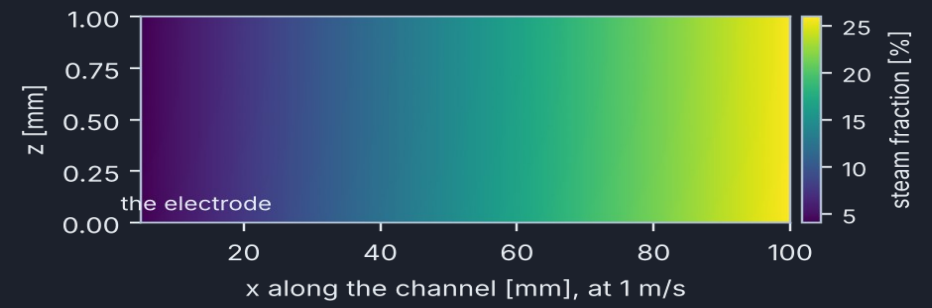
## WHERE THE CLASSICAL METHOD STAYS AHEAD

One small solve: the pipe's grid meets $10^{-3}$ in 1 ms, and a stored field can be interpolated. The network pays with many queries, a speed as an input, and higher dimensions.

## THE FIELDS THE NETWORKS LEARN




*the velocity in the tube over the 4 s after the pump starts (Ex09.1)*




*the steam fraction along the fuel channel at 1 m/s; the network beats any grid tested (Ex09.2)*

# Electrochemistry

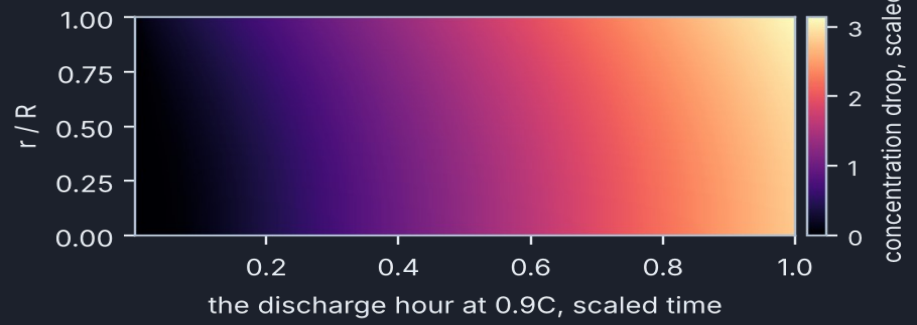
## WHERE THE NETWORK BEATS THE CLASSICAL METHOD

| | |
|---|---|
| **one network** | position, voltage, inlet temperature in |
| **accuracy** | 0.19 K against the reference's 0.40 K |
| **a thousand points** | about 500× faster, same CPU |
| **optimise through it** | the cheapest hour by gradient |
| **the particle** | diffusivity back-solved, 0.30 % off |

## WHERE THE CLASSICAL METHOD STAYS AHEAD

For one operating point the coupled finite-volume solver is still the simple choice. The network's case is the whole operating window, carried by one training of 33 minutes.

## THE FIELD THE NETWORK LEARNS




*the drop of lithium concentration in the particle along the radius, over the discharge hour at 0.9C (Ex10.1)*

**500×**

faster over a thousand operating points of the electrolysis cell, 0.19 K against the 50-cell reference's 0.40 K (Ex10.2)

# Power Grid

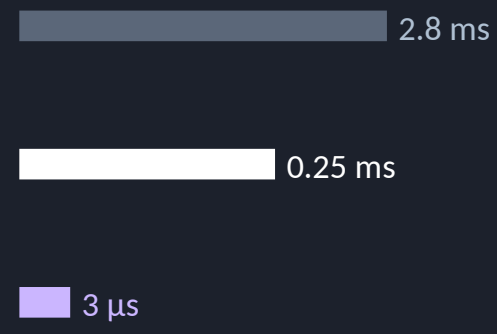
## WHERE THE NETWORK BEATS THE CLASSICAL METHOD

| | |
|---|---|
| **state estimation** | least-squares accuracy in 0.25 ms |
| **inertia** | 3.89 s found, true 4.00, both starts |
| **screening** | a verdict in 3 µs a case |
| **new operating points** | within 0.6 ms of the simulation |

## WHERE THE CLASSICAL METHOD STAYS AHEAD

An unseen topology was 57 ms off, on the unsafe side: the graph network orders the queue of cases, and every unseen topology still goes to the swing simulation.

## COMPUTATION TIME, ONE CASE




*time for one answer, logarithmic scale - measured on a laptop CPU; the edge board is the mini projects' to measure*

**1000×**

the screening gain: 3 µs a verdict against 2.8 ms a simulated case

# Vision and Navigation

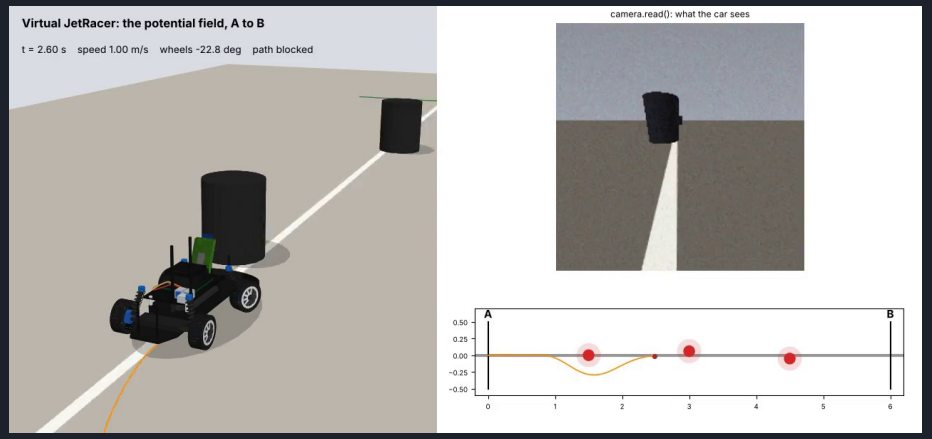
## WHERE THE NETWORK BEATS THE CLASSICAL METHOD

| | |
|---|---|
| **the race** | the rule crashes at 3.9 s; DL arrives |
| **right frames** | 95.1 % against the dark rule's 85.1 |
| **false stops** | none, against 13.5 % of free frames |
| **the lap** | within 6 cm up to 17 cm blind |

## WHERE THE CLASSICAL METHOD STAYS AHEAD

Nowhere on this task - but the grey box, as light as the floor, slips past rule and network alike. That is a limit of one camera, and the mini projects' second sensor answers it.

## THE TWIN, DRIVING




*the deep-learning JetRacer following the potential field around the obstacles, drawn by the exercise's own twin*

**8.49 s**

A to B through the obstacles, five seeds alike - the conventional car crashes at 3.9 s (Ex11)

# PINN Fundamentals

## THE PROJECTS

| | |
|---|---|
| **MP7.1A** | the slot with its insulation |
| **MP7.1B** | how old is the insulation? |
| **MP7.2A** | a die with a moving hot spot (optional set) |
| **MP7.2B** | where was the panel struck? (optional set) |
| **MP7.2C** | the eigenvalues of a real slot (optional set) |

*The pattern of every pair in Part 2: A forward on richer physics than the set, B inverse from committed readings.*

## THE BAR, PER PROJECT

| | |
|---|---|
| **MP7.1A** | hot spot within 0.1 K, 7 points |
| **MP7.1B** | the age, from the readings |
| **MP7.2x** | the optional Helmholtz set's three |

### GROUND TRUTH

Built by tools/miniprojects/ex071_truth.py and ex072_truth.py, committed privately; an inverse project's answers stay back for grading.

# Heat Transfer

## THE PROJECTS

| | |
|---|---|
| **MP8.1A** | the fins resolved |
| **MP8.1B** | two modules on one sink |
| **MP8.2A** | thermal cycling |
| **MP8.2B** | heat capacity and coefficient together |

*All four on the heat sink of Ex08: resolved into its fins, doubled, pulsed in time, and identified from readings.*

## THE BAR, PER PROJECT

| | |
|---|---|
| **MP8.1A** | hottest 0.1 K, field 0.2 K, heat 1 % |
| **MP8.1B** | hottest 0.1 K per gap, optimum 2 mm |
| **MP8.2A** | peaks within 0.3 K, settling exact |
| **MP8.2B** | both constants within 5 % |

### GROUND TRUTH

Finite volumes on 0.25 mm cells (ex08_truth.py), committed privately; MP8.2B's two constants stay back for grading.

# Fluid Flow

## THE PROJECTS

| | |
|---|---|
| **MP9.1A** | one bend of the serpentine, in 3-D |
| **MP9.1B** | the pump's speed reference |
| **MP9.2A** | the SOEC steam channel |
| **MP9.2B** | the air channel |

*9.1 is the liquid serpentine - the bend's solver is yours to find, and 9.1B's second case turns turbulent. 9.2 is the cell's gas channels.*

## THE BAR, PER PROJECT

| | |
|---|---|
| **MP9.1A** | the solver proved 3 ways, then PINN |
| **MP9.1B** | module at most 80.2 °C, pumping 5 % |
| **MP9.2A** | speed 1 %, electrode 0.1 point |
| **MP9.2B** | electrode 0.1 point, Stefan answered |

### GROUND TRUTH

ex091_truth.py (with the Dean solver) and ex092_truth.py, committed privately; MP9.1A deliberately ships no 3-D reference.

# Electrochemistry

## THE PROJECTS

| | |
|---|---|
| **MP10.1A** | the full model, where the SPMe stops |
| **MP10.1B** | how much has the cell aged? |
| **MP10.2A** | the channel across its height |
| **MP10.2B** | a day under an hourly price |

*10.1 is the battery particle: the full model where the single-particle model stops, and the cell's age from its charging history. 10.2 is the electrolyser: the electrode resolved, and a day of operation.*

## THE BAR, PER PROJECT

| | |
|---|---|
| **MP10.1A** | voltage 20 mV at 1C, 3C; 50 at 5C |
| **MP10.1B** | both factors 5 %, capacity 1 % |
| **MP10.2A** | current 0.5 %, outlet mean 0.005 |
| **MP10.2B** | the day's cost within 0.5 % |

### GROUND TRUTH

PyBaMM sets (Prada2013, Chen2020) and ex101_truth.py / ex102_truth.py, committed privately; the ageing factors stay back for grading.

# Vision and Navigation

## THE PROJECTS

| | |
|---|---|
| **MP11A** | faster from A to B, on the ROSMASTER R2 |
| **MP11B** | camera, LiDAR and ultrasonic in difficult conditions |

*The two hardware projects: both are developed on the virtual JetRacer first - pb.lidar, pb.sonar, pb.run_course - and graded on the lab's Yahboom ROSMASTER R2, on the taped 6 m course with the lab's measured obstacle positions.*



*the Yahboom ROSMASTER R2, Superior version: Jetson Orin Nano Super 8 GB, LiDAR on the top deck, depth camera at the front (vendor photograph)*

## THE BAR, PER PROJECT

| | |
|---|---|
| **MP11A** | no collision, at most 5.0 s (twin) |
| **MP11B** | 5 % per kind, no collision (twin) |
| **the car** | five runs without contact, in the lab |

### GROUND TRUTH

ex11_truth.py: perfect perception's 8.76 s on 20 held-back courses. The car: the ROSMASTER R2 (Orin Nano, LiDAR, depth camera), a URM09 added.

# Power Grid

## THE PROJECTS

| | |
|---|---|
| **MP12.1A** | the state of the 39-bus grid |
| **MP12.1B** | the inertia of every machine |
| **MP12.2A** | contingency screening on 39 buses |
| **MP12.2B** | the same screening with converters |

*The six-bus methods, unchanged, on the IEEE 39-bus grid - nothing harder. The solved optional sets, MP12.1A-39 to MP12.2B-39, are development areas beyond these.*

## THE BAR, PER PROJECT

| | |
|---|---|
| **MP12.1A** | worst bus 0.01 p.u., beat WLS time |
| **MP12.1B** | inertias 5 %, dampings 20 % |
| **MP12.2A** | no miss, 5 % false, mean 5 ms |
| **MP12.2B** | no insecure case missed |

### GROUND TRUTH

ex121_truth.py and ex122_truth.py on grid39_classic.py: pandapower's case39 with the course's classical machine model, committed privately.

# Choosing a Project

## AN HONEST COMPARISON

| | |
|---|---|
| **fundamentals** | the cleanest physics; the slot and the Helmholtz problems reward care, not compute |
| **heat** | easiest to start; the comb geometry and the double identification carry the interest |
| **flow** | most likely to produce a negative result honestly earned; 9.1A has no reference on purpose |
| **electrochemistry** | most data-driven; PyBaMM does the heavy lifting, so the question carries it |
| **vision** | most fun, most hardware risk; the twin first, then five runs on the R2 |
| **grid** | most open-ended; the 39-bus grid is the one everyone publishes on |

### PICK BY THE QUESTION, NOT THE FIELD

The best projects come from wanting to know the answer. If none of these interests you, propose your own.

### AND START WITH THE BASELINE

Every one of these has a classical method to beat, or to lose to honestly. Build that first.

### ONE PROJECT PER TEAM OF TWO OR THREE

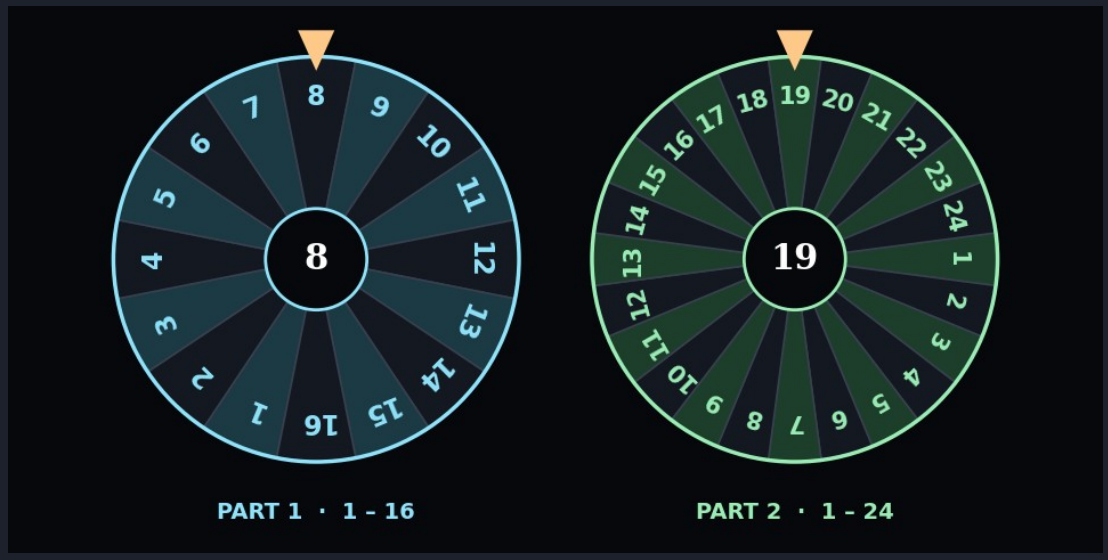
Chosen after L12.2; the chosen project's files come from the lecturer. Come and talk before you commit.

# The Exam Draw

Each student draws two numbers: **one from Part 1 (1 – 16)**, then **one from Part 2 (1 – 24)**. The number gives the question in the order of the tables that follow.

**Spin the roulette ▶**

*Opens the draw in the browser: Part 1 first, then Part 2, each a random number.*




*The two roulettes, as the draw shows them. The film is an illustration; the numbers are drawn at random only on the web page.*

# Questions 3.1 – 4.4

| draw | nr | question | from |
|---|---|---|---|
| 1 | **3.1** | How does a neural network learn the motion of a vibrating mass from measurements? | L3.1 Q1 |
| 2 | **3.2** | Why did the network that matched the measurements best predict worst? | L3.1 Q2 |
| 3 | **3.3** | What changes when the equation of motion is added to the network's loss? | L3.1 Q3 |
| 4 | **3.4** | What does a surrogate network take and return, and where can it not be trusted? | L3.2 Q3 |
| 5 | **4.1** | What does a single neuron compute from its inputs, and what can it not learn? | L4.1 Q1 |
| 6 | **4.2** | Why does a network that fits a curve need an activation function, and why must it have a slope? | L4.1 Q2 |
| 7 | **4.3** | What does a deep network gain over a wide one with the same number of weights? | L4.2 Q1 |
| 8 | **4.4** | How do the training error and the error on unseen data change as a network grows? | L4.2 Q3 |

# Questions 5.1 – 6.4

| draw | nr | question | from |
|---|---|---|---|
| 9 | **5.1** | Why does a convolutional network do better than a dense one on images it has not seen? | L5.1 Q2 |
| 10 | **5.2** | How does a graph network compute on a power grid, bus by bus? | L5.1 Q3 |
| 11 | **5.3** | How does a recurrent network read a series of measurements, such as the electricity demand? | L5.2 Q1 |
| 12 | **5.4** | Why does an LSTM remember longer than a plain recurrent network? | L5.2 Q4 |
| 13 | **6.1** | Where does a loss function come from, if it is not chosen by habit? | L6.1 Q1 |
| 14 | **6.2** | How does a network find the weights that lower its loss? | L6.1 Q3 |
| 15 | **6.3** | How do SGD, Adam and L-BFGS differ, and in which order are they used? | L6.1 Q4 |
| 16 | **6.4** | Why can a network trained on another task help with yours? | L6.2 Q1 |

# Questions 7.1 – 8.4

| draw | nr | question | from |
|---|---|---|---|
| 1 | **7.1** | How can a network be the temperature field in the slot without any solved data? | L7.1 Q1 |
| 2 | **7.2** | Why is a wall temperature built into the network more accurate than one put in the loss? | L7.1 Q3 |
| 3 | **7.3** | How does the type of the equation decide what the network must be given? | L7.2 Q1 |
| 4 | **7.4** | How are the terms of the loss balanced, and how many collocation points does the network need? | L7.1 Q4 |
| 5 | **8.1** | Why is everything scaled before the heat sink's network is trained? | L8.1 Q2 |
| 6 | **8.2** | What does the heat sink's network have built in, and what is left for it to learn? | L8.1 Q3 |
| 7 | **8.3** | Why does a network's loss in time ignore causality, and how is training steered to the start? | L8.2 Q2 |
| 8 | **8.4** | What does the network of the pulse have built in, and how does it compare with Crank–Nicolson? | L8.2 Q3 |

# Questions 9.1 – 10.4

| draw | nr | question | from |
|---|---|---|---|
| 9 | **9.1** | Why are the radius, the time and the velocity scaled before the network is trained? | L9.1 Q2 |
| 10 | **9.2** | When is the trained network faster than finite differences, and how is that judged fairly? | L9.1 Q3 |
| 11 | **9.3** | Why does the network not keep the mean steam fraction right by itself, and how is it enforced? | L9.2 Q2 |
| 12 | **9.4** | How is the inlet speed for a required hydrogen fraction found with the trained network? | L9.2 Q4 |
| 13 | **10.1** | Why does the network not conserve the lithium by itself, and how is that enforced? | L10.1 Q3 |
| 14 | **10.2** | How can the diffusivity be found with a network from surface measurements, and when can it not be found? | L10.1 Q4 |
| 15 | **10.3** | How can one network give the steam, the temperature and the current along the channel at every operating point? | L10.2 Q1 |
| 16 | **10.4** | How is the optimal cell voltage found with the trained network? | L10.2 Q3 |

# Questions 11.1 – 12.4

| draw | nr | question | from |
|---|---|---|---|
| 17 | **11.1** | Why can a convolutional network steer from a camera frame where a line detector fails? | L11.1 Q1 |
| 18 | **11.2** | What must the training frames show for the network to bring a drifting car back? | L11.1 Q2 |
| 19 | **11.3** | Why is the threshold on the probability of a blocked path not set at one half? | L11.2 Q2 |
| 20 | **11.4** | Why does a convolutional network need a GPU to steer in time? | L11.1 Q4 |
| 21 | **12.1** | Weighted least squares is already a statistical estimator: what does a network add to it? | L12.1 Q2 |
| 22 | **12.2** | How can the inertia of a machine be found with a network from a recorded disturbance? | L12.1 Q3 |
| 23 | **12.3** | Where is the physics in the graph network, if not in its loss? | L12.2 Q2 |
| 24 | **12.4** | Why is an unseen topology the fair test of the graph network, and what did it show? | L12.2 Q4 |

# Question 3.1

## How does a neural network learn the motion of a vibrating mass from measurements?

### THE START OF AN ANSWER

It takes the time and returns the position of the mass. Its weights are changed step by step until its output passes close to the measured points. It is told nothing about springs or damping; all it knows comes from five seconds of data.

### GO DEEPER

*What the loss measures (Ex_03 section 4); how the steps are taken (L6.1).*

# Question 3.2

## Why did the network that matched the measurements best predict worst?

### THE START OF AN ANSWER

It also learned the sensor's noise and bias, because it cannot tell them from the motion. Inside the five seconds it looked perfect. After them it missed by 30 mm, because nothing told it the mass keeps swinging.

### GO DEEPER

*The error inside against outside the window (Ex_03 section 4); overfitting (UDL ch. 8).*

# Question 3.3

## What changes when the equation of motion is added to the network's loss?

### THE START OF AN ANSWER

The network must now obey the equation at many instants, also where there is no measurement. It stops following noise that breaks the equation, and it predicted within 0.6 mm after the data ended.

### GO DEEPER

*The weight given to the physics (Ex_03 section 6); the same idea in L7.1.*

# Question 3.4

## What does a surrogate network take and return, and where can it not be trusted?

### THE START OF AN ANSWER

It takes the inputs of a simulation, such as a geometry or an operating point, and returns its result, learned from a few hundred runs. It answers in microseconds instead of hours, which pays in a design loop. Outside the inputs it was trained on, it has no reason to be right.

### GO DEEPER

*Ex_03 section 4, after the data ends; L12.2's screen, a surrogate for a simulation.*

# Question 4.1

## What does a single neuron compute from its inputs, and what can it not learn?

### THE START OF AN ANSWER

It weighs each input, adds a bias and passes the sum through an activation function. So it draws one straight boundary: it can learn AND or OR, but not XOR. Linear regression is one neuron with no activation.

### GO DEEPER

*The perceptron on the logic gates (Ex_04 notebook 01); UDL ch. 3.1.*

# Question 4.2

## Why does a network that fits a curve need an activation function, and why must it have a slope?

### THE START OF AN ANSWER

Without one, every layer is a weighted sum, and a sum of sums is still a straight line, however large the network. Training follows the slope of the loss, so the activation must have a slope; the perceptron's step has none.

### GO DEEPER

*Training with and without it (Ex_04 notebook 02); GBC ch. 6.1.*

# Question 4.3

## What does a deep network gain over a wide one with the same number of weights?

### THE START OF AN ANSWER

Each layer folds the output of the layer before, so the number of straight pieces multiplies with depth, while width only adds to it. For the same number of weights a deep network can describe a more detailed function.

### GO DEEPER

*Deep against wide (Ex_04 notebook 04); UDL ch. 4.*

# Question 4.4

## How do the training error and the error on unseen data change as a network grows?

### THE START OF AN ANSWER

The training error keeps falling. The error on unseen data falls, then rises once the network starts to fit the noise. A curve through every training point with a poor error on unseen data is an overfit.

### GO DEEPER

*Both errors against the size of the network (Ex_04 notebook 05); UDL ch. 8.1 – 8.2.*

# Question 5.1

## Why does a convolutional network do better than a dense one on images it has not seen?

### THE START OF AN ANSWER

One kernel is shared by every position, so it has far fewer weights, and a pattern learned in one place is found in every other. In Ex_05 it scored 0.99 on unseen images, the dense network 0.71.

### GO DEEPER

*Shift an image and compare both networks (Ex_05 notebook 01); UDL ch. 10.2.*

# Question 5.2

## How does a graph network compute on a power grid, bus by bus?

### THE START OF AN ANSWER

In each layer every bus collects the numbers of the buses it is connected to, combines them with its own through shared weights, and is updated. After k layers a bus has heard from every bus within k lines.

### GO DEEPER

*One layer on the six-bus grid (Ex_05 notebook 02); UDL ch. 13.1 – 13.4.*

# Question 5.3

## How does a recurrent network read a series of measurements, such as the electricity demand?

### THE START OF AN ANSWER

One value at a time, keeping a hidden state as its memory. At each step it updates the state from the new value and the old state, always with the same weights, and gives a forecast from the state.

### GO DEEPER

*The forecast of the demand (Ex_05 notebook 04); GBC ch. 10.1 – 10.2.*

# Question 5.4

## Why does an LSTM remember longer than a plain recurrent network?

### THE START OF AN ANSWER

In a plain one the gradient is multiplied by the same weights at every step and fades over a long series. The LSTM updates its cell state by adding, so the gradient passes along it nearly unchanged. In Ex_05 it had the lowest error on unseen data.

### GO DEEPER

*Compare the three networks (Ex_05 notebook 04); GBC ch. 10.7.*

# Question 6.1

## Where does a loss function come from, if it is not chosen by habit?

### THE START OF AN ANSWER

From the noise in the measurements. Choose a distribution for the noise, let the network predict its parameters, and the loss is minus the log of the probability of the data. Squared error is what Gaussian noise gives.

### GO DEEPER

*The noise of the data in Ex_06 notebook 01; UDL ch. 5.1 – 5.2.*

# Question 6.2

## How does a network find the weights that lower its loss?

### THE START OF AN ANSWER

By gradient descent: small steps of every weight in the direction that lowers the loss. Backpropagation gives all the gradients in one backward pass, at about the cost of one more forward pass, and autograd writes it for you.

### GO DEEPER

*Backpropagation by hand on a small network (UDL ch. 7.2 – 7.4); the step size (UDL ch. 6.1).*

# Question 6.3

## How do SGD, Adam and L-BFGS differ, and in which order are they used?

### THE START OF AN ANSWER

SGD takes the gradient on a random batch, Adam scales each weight's step by its own history, and L-BFGS also uses the curvature. Tune the learning rate first. In Ex_06, Adam followed by L-BFGS ended about twenty times lower than Adam alone.

### GO DEEPER

*The race of the optimisers (Ex_06 notebook 02); UDL ch. 6.2 – 6.5.*

# Question 6.4

## Why can a network trained on another task help with yours?

### THE START OF AN ANSWER

Its first layers hold general features, such as edges and textures, that carry over. They are kept, and only a new last layer is trained on your data. With few labels this starts far ahead of random weights.

### GO DEEPER

*Transfer with few labels (Ex_06 notebook 03); UDL ch. 9.3.6.*

# Question 7.1

## How can a network be the temperature field in the slot without any solved data?

### THE START OF AN ANSWER

It takes the position and the current and returns the temperature rise. Autograd gives its derivatives, and the loss is the residual of the conduction equation at points inside the slot, plus the wall conditions. No solution is needed as a label.

### GO DEEPER

*The terms of the loss (Ex_07.1 section 4); Liu ch. 2.1 – 2.2.*

# Question 7.2

## Why is a wall temperature built into the network more accurate than one put in the loss?

### THE START OF AN ANSWER

In the loss the wall condition competes with the equation and holds only roughly. Multiplied by a mask that is zero on the walls, the network meets the condition for any weights. Built in: 0.02 K; in the loss: about 1.5 K off near the walls.

### GO DEEPER

*Both versions (Ex_07.1 sections 4 – 5); Liu ch. 2.5.3.*

# Question 7.3

## How does the type of the equation decide what the network must be given?

### THE START OF AN ANSWER

Poisson's equation has no time: the position only, and a condition on every wall. The heat equation adds time and one initial field; the wave equation needs two, the field and the velocity. With one missing, the loss still reaches zero, but at a wrong field.

### GO DEEPER

*The panel without its initial velocity (Ex_07.2 notebooks 1 – 3); Liu ch. 2.*

# Question 7.4

## How are the terms of the loss balanced, and how many collocation points does the network need?

### THE START OF AN ANSWER

The terms differ in units and size, and a small one is neglected. Scale the variables to order one, then raise a weight by hand or from the sizes of the gradients. The equation holds only at the points: use ten to twenty times the number of weights.

### GO DEEPER

*Change the weights and the points (Ex_07.1 section 4); Liu ch. 2.2.3, 2.6.*

# Question 8.1

## Why is everything scaled before the heat sink's network is trained?

### THE START OF AN ANSWER

In metres and kelvin the loss terms differ by orders of magnitude, while the network's starting weights and the loss weights assume order one. Ex_08 divides the position by the size of the base, the temperature by the hand calculation's rise, and the residual by the small Biot number.

### GO DEEPER

*The scaled equation (Ex_08 section 3); Liu ch. 4.4.*

# Question 8.2

## What does the heat sink's network have built in, and what is left for it to learn?

### THE START OF AN ANSWER

The one-dimensional hand calculation through the base, which holds most of the answer; the network adds only the sideways spreading under the module. Its input is a cosine of the position, so the insulated ends hold for any weights.

### GO DEEPER

*The trial function (Ex_08 section 5); Liu ch. 2.2.*

# Question 8.3

## Why does a network's loss in time ignore causality, and how is training steered to the start?

### THE START OF AN ANSWER

The loss is one mean over space and time, so a late instant counts as much as an early one, and the network may fit the end first. The remedies: build the start in, weight each instant by how well its past fits, train window by window, or sample early times densely.

### GO DEEPER

*Where Ex_08 puts its points (section 11); Liu ch. 4.4.*

# Question 8.4

## What does the network of the pulse have built in, and how does it compare with Crank–Nicolson?

### THE START OF AN ANSWER

The lumped model of the whole sink, an exact start and insulated ends; the network learns only the spreading. Its clock is the square root of time, which stretches the first seconds. Crank–Nicolson was within hundredths of a kelvin, the network one to two kelvin off during the pulse.

### GO DEEPER

*Ex_08 sections 9 – 11; Liu ch. 4.2 – 4.4.*

# Question 9.1

## Why are the radius, the time and the velocity scaled before the network is trained?

### THE START OF AN ANSWER

The radius is millimetres, the window seconds, and the second derivative tens of thousands of times the velocity. Neurons with tanh train well on numbers of order one. Scaled, two numbers carry the physics: the scaled viscosity and the push of the pressure gradient.

### GO DEEPER

*The scaled equation (Ex_09.1 sections 3, 5); what changes unscaled.*

# Question 9.2

## When is the trained network faster than finite differences, and how is that judged fairly?

### THE START OF AN ANSWER

Online against online: one solve against one evaluation, on the same device, at the same accuracy, with the training counted apart. In this small tube the gain is tens, and finite differences answer first. The network pays for many new cases, a parameter as input, or more dimensions.

### GO DEEPER

*The timing and its scaling with dimension (Ex_09.1 section 5).*

# Question 9.3

## Why does the network not keep the mean steam fraction right by itself, and how is it enforced?

### THE START OF AN ANSWER

The built-in hand calculation gives the right mean, but the network can add a mean of its own. A loss term holds the flow-weighted mean of the network's part at zero. With it the error falls from about a ten-thousandth to a few millionths.

### GO DEEPER

*Train without the balance (Ex_09.2 section 5); Incropera ch. 8.*

# Question 9.4

## How is the inlet speed for a required hydrogen fraction found with the trained network?

### THE START OF AN ANSWER

Autograd through the network gives the derivative of the hydrogen fraction at the outlet in the speed, and Newton's method finds 0.3173 m/s against the exact 0.3174 in five steps. It must start near the answer: from the hand calculation it converges, from 1 m/s it diverges.

### GO DEEPER

*Newton from two starts (Ex_09.2 section 6); Liu ch. 2.2.*

# Question 10.1

## Why does the network not conserve the lithium by itself, and how is that enforced?

### THE START OF AN ANSWER

Finite volumes conserve it by construction. A network obeys the equation only roughly, and small errors in the surface flux add up over the discharge. The amount that has entered is known from the current, so a loss term holds the mean concentration to it.

### GO DEEPER

*Switch the term off (Ex_10.1 section 5); Liu ch. 2.*

# Question 10.2

## How can the diffusivity be found with a network from surface measurements, and when can it not be found?

### THE START OF AN ANSWER

It becomes a trainable number, fitted with the network to the equation and the measurements. It is identifiable because the surface lags behind the mean by an amount that depends on it. On the flat voltage plateau of LFP the voltage barely changes, and no method can find it.

### GO DEEPER

*The inverse problem (Ex_10.1 section 6); Liu ch. 4.*

# Question 10.3

## How can one network give the steam, the temperature and the current along the channel at every operating point?

### THE START OF AN ANSWER

It takes the position, the cell voltage and the inlet temperature and returns the three fields, trained over the whole operating range until the three laws hold. The inlet values are built in. Any operating point is then read off, not solved again.

### GO DEEPER

*The network and its inputs (Ex_10.2 section 5); Liu ch. 2.2.*

# Question 10.4

## How is the optimal cell voltage found with the trained network?

### THE START OF AN ANSWER

The cost of an hour, the electricity less the hydrogen plus the ageing, is read from the fields, with the 10 K limit as a penalty. Autograd gives its derivative in the voltage, and gradient descent finds 1.250 V in seconds; a sweep of 81 solver runs agrees.

### GO DEEPER

*With and without the limit (Ex_10.2 section 6); Liu ch. 5.*

# Question 11.1

## Why can a convolutional network steer from a camera frame where a line detector fails?

### THE START OF AN ANSWER

The detector follows fixed rules, so anything bright pulls its aim. The network learns its own features from frames with the true position of the line. With glare the detector's error rose to 0.08, while the network's stayed at 0.05.

### GO DEEPER

*The worst frames of each (Ex_11 sections 3 – 4); UDL ch. 10.*

# Question 11.2

## What must the training frames show for the network to bring a drifting car back?

### THE START OF AN ANSWER

The floor, the light and the disturbances it will meet, and positions off the line, labelled with the way back. So the driver who records them swerves on purpose. Trained on a smooth drive, the network is about five times worse beside the line.

### GO DEEPER

*The smooth against the swerving drive (Ex_11 section 4); UDL ch. 8.*

# Question 11.3

## Why is the threshold on the probability of a blocked path not set at one half?

### THE START OF AN ANSWER

A missed obstacle costs a collision, a false stop a second. Lowering the threshold trades misses for false stops; choose it where the expected cost is smallest. At one half the network missed 15 % of the obstacles, at 0.05 it missed 4 % and stopped for 5 % of the free frames.

### GO DEEPER

*The cost against the threshold (Ex_11 section 5); measure both rates on the car.*

# Question 11.4

## Why does a convolutional network need a GPU to steer in time?

### THE START OF AN ANSWER

Every output of a convolution is its own sum of products, independent of the others, so a GPU's many cores compute them at once. The exercise's network takes a millisecond on a laptop, inside the 33 ms frame; a ResNet-18 does a hundred times the work and fits only on a GPU.

### GO DEEPER

*Count the multiply-adds (Ex_11 section 7); UDL ch. 10.*

# Question 12.1

## Weighted least squares is already a statistical estimator: what does a network add to it?

### THE START OF AN ANSWER

Both minimise the same weighted misfit, so from the same readings they are equally accurate. Least squares solves it anew for every set of readings by Newton iterations; the network learns it once. Its gain is time, a quarter of a millisecond, and its misfit flags a line out.

### GO DEEPER

*The line-out test (Ex_12.1 sections 4 – 5); Liu ch. 4.*

# Question 12.2

## How can the inertia of a machine be found with a network from a recorded disturbance?

### THE START OF AN ANSWER

The network takes the time after the fault and returns the angle and the speed; the inertia is one more trainable number. The loss is the residual of the swing equation plus the misfit to the record. The inertia must multiply the acceleration; dividing by it lets a huge value hide.

### GO DEEPER

*The inverse problem (Ex_12.1 section 6); Liu ch. 4.*

# Question 12.3

## Where is the physics in the graph network, if not in its loss?

### THE START OF AN ANSWER

The loss is only the squared error to the simulated clearing time. The physics is in the labels, from the swing equation, and in the graph the network is given. It learns how the clearing time varies on the grids it has seen; it does not solve the swing equation.

### GO DEEPER

*The training (Ex_12.2 section 5); UDL ch. 13.*

# Question 12.4

## Why is an unseen topology the fair test of the graph network, and what did it show?

### THE START OF AN ANSWER

Outages seen in training test interpolation: there it was within 0.6 ms. With line 0 out never shown, it was 57 ms off on the unsafe side and missed all eleven insecure cases. The ranking held, but the threshold that rescued it was found with the truth known.

### GO DEEPER

*The unseen line (Ex_12.2 section 6); what that means in a control room.*